Team Project

Team Information

Team Lead Email: tjv55@nau.edu

Team Members:

1. Tyler Vander Mooren
2. Thomas Peaslee

Project Details

Selected Database Platform:

■ Cassandra

Selected Project Name: IoT Sensor Management System

Completion Confirmation

By submitting this form, I confirm on behalf of my team that:

■ **Project Implementation is Complete**
All required technical components have been implemented and tested according to the project specifications.

■ **Documentation is Finalized**
All technical documentation, setup instructions, and team process documentation are complete and ready for review.

■ **Database Access for Instructor Confirmed**
The professor has been granted appropriate access to our database instance with the following details:

Database Access Information:

- Platform/Service Used: Astra Databases
- Connection Method: Astra.Database.com – CQL Console
- Professor Username/Account: itc413team4

{
  "clientId": "ggmrQNiysOilzsdYPrzZvBLh",

"secret": "IvHU-.th2v6Pbi.oB+FnnJkKdBC0k_izE2MrqE+OdKzqSjKNq-2cPImZ_Om.QMDLij0,pj-uGGcvruQ3Hl_qekB51sn1yZ+Z-9R3LjIU5CH68GY7fHZFQA-5C.hHcAzC",
"token": "AstraCS:ggmrQNiysOilzsdYPrzZvBLh:337730dbab1b5fd67a6b8883aca35cd77d087c03125446b16da56d800e5f2f0d"
}

- Access Level Granted: ■ Read-Only ☐ Full Access ☐ Admin
- Date Access Granted: April 29, 2026
- Backup Access (if token doesn't work, this is login to astra page)
- Username: itc413team4
- Password: Team4capstone!
- Access Level Granted: ■ Admin

Supporting Technical Documentation

Screen capture of Schema/Data Model and a list of who was involved and description of their role in design and implementation of the data model in the project database:

```
token@cqlsh:iot_sensor> DESCRIBE TABLES;

daily_aggregates_by_region  latest_readings_by_region
device_registry_by_sensor   sensor_readings_by_sensor_day
devices_by_region_status    violations_by_region_day

token@cqlsh:iot_sensor> DESCRIBE TABLE sensor_readings_by_sensor_day;

CREATE TABLE iot_sensor.sensor_readings_by_sensor_day (
    sensor_id text,
    reading_date date,
    reading_timestamp timestamp,
    reading_value double,
    region_id text,
    sensor_type text,
    status text,
    threshold_flag boolean,
    unit text,
    PRIMARY KEY ((sensor_id, reading_date), reading_timestamp)
) WITH CLUSTERING ORDER BY (reading_timestamp DESC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.UnifiedCompactionStrategy'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Tyler designed and implemented the main Cassandra data model, including the query-focused tables for sensor readings, latest readings, threshold violations, device registry records and daily aggregates. Thomas assisted with the project completion documentation.

Screen capture of data populating the database and a list of who was involved and a description of their role in creation of the data and input into the project database:

```
token@cqlsh:iot_sensor> SELECT * FROM device_registry_by_sensor LIMIT 10;

 sensor_id   | install_date | last_maintenance_date | location_name            | normal_max | normal_min | region_id | sensor_name                    | sensor_type   | status      | unit
-------------+--------------+-----------------------+--------------------------+------------+------------+-----------+--------------------------------+---------------+-------------+------------------
   N-AQI-001 |   2026-01-12 |            2026-04-01 |    North Industrial Zone |        100 |          0 |     North |      North Air Quality Sensor 1 |   air_quality |      ACTIVE |              AQI
  N-TRAF-001 |   2026-01-14 |            2026-04-02 |          North Main Road |       1200 |          0 |     North | North Traffic Volume Sensor 1 | traffic_volume |      ACTIVE | vehicles_per_hour
 N-SPEED-001 |   2026-01-16 |            2026-04-02 |       North Highway Exit |         75 |         25 |     North |          North Speed Sensor 1 |     avg_speed |     OFFLINE |              mph
  S-TRAF-001 |   2026-01-30 |            2026-04-06 |          South Main Road |       1200 |          0 |     South | South Traffic Volume Sensor 1 | traffic_volume |      ACTIVE | vehicles_per_hour
  S-TEMP-001 |   2026-01-26 |            2026-04-05 |               South Gate |         95 |         32 |     South |    South Temperature Sensor 1 |   temperature |      ACTIVE |                F
 S-SPEED-001 |   2026-02-01 |            2026-04-06 |       South Highway Exit |         75 |         25 |     South |          South Speed Sensor 1 |     avg_speed |     OFFLINE |              mph
   C-AQI-001 |   2026-01-20 |            2026-04-03 | Central Business District |       100 |          0 |   Central |    Central Air Quality Sensor 1 |   air_quality |      ACTIVE |              AQI
  N-TEMP-001 |   2026-01-10 |            2026-04-01 |               North Gate |         95 |         32 |     North |    North Temperature Sensor 1 |   temperature |      ACTIVE |                F
  C-TEMP-001 |   2026-01-18 |            2026-04-03 |          Central Station |         95 |         32 |   Central |  Central Temperature Sensor 1 |   temperature |      ACTIVE |                F
  C-TRAF-001 |   2026-01-22 |            2026-04-04 |        Central Main Road |       1200 |          0 |   Central | Central Traffic Volume Sensor 1 | traffic_volume | MAINTENANCE | vehicles_per_hour

(10 rows)
token@cqlsh:iot_sensor> SELECT * FROM sensor_readings_by_sensor_day
            ... WHERE sensor_id = 'N-TEMP-001'
            ... AND reading_date = '2026-04-21';

 sensor_id  | reading_date | reading_timestamp               | reading_value | region_id | sensor_type | status | threshold_flag | unit
------------+--------------+---------------------------------+---------------+-----------+-------------+--------+----------------+------
 N-TEMP-001 |   2026-04-21 | 2026-04-21 14:00:00.000000+0000 |          97.2 |     North | temperature | ACTIVE |           True |    F
 N-TEMP-001 |   2026-04-21 | 2026-04-21 12:00:00.000000+0000 |          88.7 |     North | temperature | ACTIVE |          False |    F
 N-TEMP-001 |   2026-04-21 | 2026-04-21 10:00:00.000000+0000 |          74.1 |     North | temperature | ACTIVE |          False |    F
 N-TEMP-001 |   2026-04-21 | 2026-04-21 08:00:00.000000+0000 |          68.4 |     North | temperature | ACTIVE |          False |    F

(4 rows)
token@cqlsh:iot_sensor> SELECT * FROM daily_aggregates_by_region
            ... WHERE region_id = 'North';

 region_id | aggregate_date | sensor_type | average_value | max_value | min_value | missing_reading_count | reading_count | threshold_violation_count
-----------+----------------+-------------+---------------+-----------+-----------+-----------------------+---------------+---------------------------
     North |     2026-04-21 | air_quality |           100 |       118 |        82 |                     0 |             2 |                         1
     North |     2026-04-21 | temperature |          82.1 |      97.2 |      68.4 |                     0 |             4 |                         1

(2 rows)
```

Tyler created and loaded the main sample data for sensors, readings, violations, latest readings, and aggregate reports. Thomas reviewed the database setup and contributed to final submission preparation.

Screen capture of list of queries and a list of who was involved in creating them, and description of their role in implementing them in the project database:

```
token@cqlsh:iot_sensor> SELECT sensor_id, reading_timestamp, region_id, sensor_type,
                    ...        reading_value, unit, status, threshold_flag
                    ... FROM sensor_readings_by_sensor_day
                    ... WHERE sensor_id = 'N-TEMP-001'
                    ... AND reading_date = '2026-04-21';

 sensor_id  | reading_timestamp               | region_id | sensor_type | reading_value | unit | status | threshold_flag
------------+---------------------------------+-----------+-------------+---------------+------+--------+----------------
 N-TEMP-001 | 2026-04-21 14:00:00.000000+0000 |     North | temperature |          97.2 |    F | ACTIVE |           True
 N-TEMP-001 | 2026-04-21 12:00:00.000000+0000 |     North | temperature |          88.7 |    F | ACTIVE |          False
 N-TEMP-001 | 2026-04-21 10:00:00.000000+0000 |     North | temperature |          74.1 |    F | ACTIVE |          False
 N-TEMP-001 | 2026-04-21 08:00:00.000000+0000 |     North | temperature |          68.4 |    F | ACTIVE |          False

(4 rows)
token@cqlsh:iot_sensor> SELECT region_id, reading_date, reading_timestamp, sensor_id,
                    ...        sensor_type, reading_value, unit, threshold_type,
                    ...        normal_min, normal_max, status
                    ... FROM violations_by_region_day
                    ... WHERE region_id = 'North'
                    ... AND reading_date = '2026-04-21';

 region_id | reading_date | reading_timestamp               | sensor_id  | sensor_type | reading_value | unit | threshold_type | normal_min | normal_max | status
-----------+--------------+---------------------------------+------------+-------------+---------------+------+----------------+------------+------------+--------
     North |   2026-04-21 | 2026-04-21 14:00:00.000000+0000 | N-TEMP-001 | temperature |          97.2 |    F |      ABOVE_MAX |         32 |         95 | ACTIVE
     North |   2026-04-21 | 2026-04-21 13:00:00.000000+0000 |  N-AQI-001 | air_quality |           118 |  AQI |      ABOVE_MAX |          0 |        100 | ACTIVE

(2 rows)
```

```
token@cqlsh:iot_sensor> SELECT region_id, sensor_id, reading_timestamp, sensor_type,
                    ...        reading_value, unit, status, threshold_flag
                    ... FROM latest_readings_by_region
                    ... WHERE region_id = 'North';

 region_id | sensor_id   | reading_timestamp               | sensor_type    | reading_value | unit              | status  | threshold_flag
-----------+-------------+---------------------------------+----------------+---------------+-------------------+---------+----------------
     North |   N-AQI-001 | 2026-04-21 13:00:00.000000+0000 |    air_quality |           118 |               AQI |  ACTIVE |           True
     North | N-SPEED-001 | 2026-04-21 07:00:00.000000+0000 |      avg_speed |             0 |               mph | OFFLINE |           True
     North |  N-TEMP-001 | 2026-04-21 14:00:00.000000+0000 |    temperature |          97.2 |                 F |  ACTIVE |           True
     North |  N-TRAF-001 | 2026-04-21 11:00:00.000000+0000 | traffic_volume |           980 | vehicles_per_hour |  ACTIVE |          False

(4 rows)
token@cqlsh:iot_sensor> SELECT region_id, aggregate_date, sensor_type, reading_count,
                    ...        average_value, min_value, max_value,
                    ...        threshold_violation_count, missing_reading_count
                    ... FROM daily_aggregates_by_region
                    ... WHERE region_id = 'North';

 region_id | aggregate_date | sensor_type | reading_count | average_value | min_value | max_value | threshold_violation_count | missing_reading_count
-----------+----------------+-------------+---------------+---------------+-----------+-----------+---------------------------+-----------------------
     North |     2026-04-21 | air_quality |             2 |           100 |        82 |       118 |                         1 |                     0
     North |     2026-04-21 | temperature |             4 |          82.1 |      68.4 |      97.2 |                         1 |                     0

(2 rows)
token@cqlsh:iot_sensor> SELECT region_id, status, sensor_type, sensor_id,
                    ...        sensor_name, location_name, unit
                    ... FROM devices_by_region_status
                    ... WHERE region_id = 'North'
                    ... AND status = 'OFFLINE';

 region_id | status  | sensor_type | sensor_id   | sensor_name          | location_name      | unit
-----------+---------+-------------+-------------+----------------------+--------------------+------
     North | OFFLINE |   avg_speed | N-SPEED-001 | North Speed Sensor 1 | North Highway Exit |  mph

(1 rows)
```

Main implementation was carried out by Tyler for the core query patterns and then testing on the project queries was done by both Thomas and Tyler. The queries demonstrate time-series retrieval, threshold violation lookup, latest regional readings, daily aggregation reports, and communication failure/status lookup. Thomas Peaslee assisted with final review and access setup.

Final Checklist

I confirm that our team project meets the following criteria:

■ All core functionality requirements are implemented and working
■ Database performs within specified response time requirements

■ All team members contributed equitably to the project
■ Code follows best practices and is well-documented
■ Database is accessible for instructor evaluation

Submission Declaration

Team Lead Name: Tyler Vander Mooren **Date:** 4/29/2026

By uploading this document, I certify that:

- This submission represents the original work of our team
- All team members have reviewed and approved this submission
- The information provided is accurate and complete
- Our team is ready for project evaluation and presentation

Additional Notes

Change of Scope:

The project scope was reduced with a two-person team in mind. The final implementation focuses on three core entities: Sensor Readings, Device Registry and Daily Aggregates. Advanced features such as anomaly detection, predictive maintenance, correlation between the related sensors, compliance reporting and full alert notification workflows were intentionally removed from the scope.

Regarding data quality the implementation has multiple features. First, readings include a threshold_flag field and Threshold violations are stored in a separate query table for efficient review. In addition, the device status values such as ACTIVE, OFFLINE and MAINTENANCE which support communication failure tracking. Lastly, daily aggregates include missing_reading_count which can support basic missing data review.

The last thing to touch on is time-series data which is bucketed by reading_date and sensor_id to keep the partitions bounded and support efficient range queries. This is done to mirror a production IoT system, where raw readings could be retained for a shorter period using TTL or archived after a retention window, while daily aggregate records could be retained longer for reporting and analytics.

Original Project Scope and Reduced Implementation

Project 2: IoT Sensor Management System

Task: Create an IoT sensor monitoring and analytics system with comprehensive time-series handling.

Technical Requirements:

Core Data Entities (Implement 4-5) -> (Implement 2-3, downsize)

Keeping:

- Sensor Readings: High-frequency time-series measurements
- Device Registry: Sensor metadata, location, configuration
- Daily Aggregates: Pre-computed daily/hourly statistics

Cutting:

- Alert Conditions: Threshold monitoring and notification system
- Maintenance Schedule: Device service tracking and predictions

Reasoning:

This should help keep the project smaller, side note: If extra time we can store alert conditions in threshold values inside the Device Registry. Same with the maintenance schedule this could be reduced to a few fields in the Device Registry: like last maintenance date, next maintenance data and service status. Again, this is just additional information, mainly if extra time is available.

Essential Query Patterns (Implement 8-10) -> (Implement 4-5, downsize)

Keeping:

1. Get latest reading from each sensor in a region.
2. Retrieve time range data from a specific sensor
3. Find sensors with readings above/below thresholds.
4. Generate daily aggregation reports.

If there is extra time:

5. Identify sensors with communication failures.

Cutting:

6. Calculate rolling averages over time windows.
7. Track device maintenance history and schedules.
8. Correlate readings between related sensors.

9. Detect anomalies and trending patterns.
10. Generate compliance and regulatory reports.

Reminder of items cut from scope, do not include:

- Correlation between related sensors
- Anomaly detection
- Trending pattern analysis
- Compliance/regulatory reporting
- Any social graph idea unless we really need it

For Instructor Use Only

Submission Received:

Database Access Verified:

Project Rubric:

☐ MongoDB Project Rubric

☐ Neo4j Project Rubric

☐ Redis Project Rubric

☐ Cassandra Project Rubric

Initial Review Comments:

Project Score:

Evaluated by: